Action-Conditioned Predictive Diagnostics for JEPA World Models

Author names to be supplied for arXiv v1

July 2026

Abstract

Latent predictive world models avoid reconstructing pixels, but this does not guarantee robust predictions when visual appearance changes without a change in state. Noise, blur, or reduced resolution may alter the encoded history; the resulting error can then grow as the model predicts forward and a planner acts on those predictions. We introduce Action-Conditioned Predictive Consistency (ACPC), a diagnostic for frozen checkpoints. ACPC encodes clean and perturbed views of the same history, predicts both forward under the same actions, and measures the distance between the two latent trajectories. Exact samplewise inequalities relate this distance to changes in multi-step prediction error and planner candidate costs. A collapsed representation would make ACPC artificially small, so we complement it with two checkpoint-level summaries: Action-Conditioned Trajectory Radius (ATR) measures sensitivity to same-state visual perturbations, and Selective Margin Pass Rate (SMPR) tests whether pairs with different logged task states remain separated. Across four LeWM control tasks, eight-step ACPC under the recorded actions predicts perturbation-induced error changes with $51.3 \pm 3.5\%$ lower cross-validated MAE than the strongest of three same-horizon controls with destroyed action information. ACPC also improves cross-task prediction of CEM cost movement and decision regret. ATR–SMPR thresholds calibrated on source tasks screen checkpoints from tasks excluded from calibration, order checkpoint pairs correctly under blur and resize at the tested severities, and apply to PLDM after model-family recalibration.

Keywords: visual world models; predictive stability; action-conditioned diagnostics; calibration transfer; visual corruption.

Code and data release: `https://github.com/Anguo-star/lewm-acpc-diagnostics`

1 Introduction

World models support control by predicting how candidate actions change the environment [11, 12]. Joint-embedding predictive architectures (JEPAs) move this prediction problem from pixels to learned representations [16, 1, 3]. By avoiding pixel reconstruction, they can reduce pressure to model nuisance detail [28, 17]. This is appealing for visual control, where lighting, texture, or image quality may change even when the physical situation does not. Latent prediction alone, however, does not establish that a trained world model will produce stable action-conditioned predictions under such changes.

For control, whether a visual difference is irrelevant depends on the state and the actions being considered. A background texture can be ignored, whereas a contact pose, doorway state, or object–goal relation may determine which action is useful. Encoder invariance cannot resolve this distinction on its own because a planner does not act directly on the encoder output. It rolls the representation forward under candidate actions and ranks the resulting predictions with a cost. A small encoder discrepancy may grow through this computation, whereas a larger one may contract before it affects the cost. This motivates the central question: after the same action sequence is applied, do clean and perturbed views of one trajectory still produce similar predicted futures?

We first use a local geometric comparison to make the problem concrete. For each clean history, we generate several perturbed views and ask how far their representations spread from the clean representation. We compare this spread with the distance to the nearest other clean history, both at the encoder output and after prediction. This comparison shows whether visual perturbations contract or expand in representation space. It is only descriptive, however: the nearest clean history need not define a task-relevant boundary, and the final predicted step does not describe the full rollout. We therefore need a controlled comparison that follows one history under one shared action sequence.

Action-Conditioned Predictive Consistency (ACPC) provides this comparison. It predicts forward from clean and perturbed views of the same history under the same actions, then measures the distance between the two predicted latent trajectories. The score requires neither an observed future nor a task outcome. Two exact inequalities connect it to quantities that do. First, when both predictions are evaluated against the same logged future, ACPC bounds the change in multi-step prediction error. Second, for LeWM's squared latent goal cost, it bounds the movement of each candidate cost and provides fixed-pool stability certificates based on the clean candidate margins.

A small ACPC value is useful only if the representation still distinguishes different states. For example, mapping every observation to a constant would make all paired rollouts identical but discard all state information. We therefore summarize each checkpoint with two complementary quantities. Action-Conditioned Trajectory Radius (ATR) reports a high quantile of the normalized ACPC values across logged evaluation windows. Selective Margin Pass Rate (SMPR) reports how often pairs labeled as different by task-state coordinates remain outside that same-state radius by a fixed margin. SMPR is a deliberately limited guard against constant-representation collapse: it tests only the supplied coordinate labels, not every semantic or action-relevant distinction. We combine ATR and SMPR into a calibrated score for screening frozen checkpoints. ACPC is an evaluation tool, not a training objective.

We evaluate this diagnostic on LeWM [19] across TwoRoom, PushT, Reacher, and Cube, using Gaussian observation noise for calibration. On the unaugmented checkpoints, eight-step ACPC under the recorded actions predicts the perturbation-induced change in prediction error about 51% better, in cross-validated MAE, than the strongest control with the same rollout length but destroyed action information (Section 4.4). ACPC computed along planner candidates consistently improves prediction of CEM decision regret (Section 4.5). Thresholds selected on source tasks screen checkpoints from tasks excluded from calibration with balanced accuracy 0.86–0.90 across all cross-task calibration splits (Section 4.6), and the calibrated score orders checkpoint pairs correctly under blur and resize at the tested severities (Section 4.8). The same analysis applies to PLDM after model-family recalibration (Section 4.7).

The contributions are:

1. **A paired-rollout diagnostic** (Section 3). ACPC measures how same-state visual changes propagate through action-conditioned predictions; ATR summarizes the checkpoint-level radius, and SMPR guards against constant-representation collapse.
2. **Exact links to prediction and planning quantities** (Propositions 1 and 2). Samplewise inequalities connect rollout disagreement to common-future error drift and to squared latent goal-cost movement.
3. **Controlled evidence beyond the calibration tasks** (Section 4). Experiments show that ACPC predicts perturbation-induced error changes and CEM decision regret, and that calibrated thresholds transfer to tasks excluded from calibration, to blur and resize, and to a second model family.

2 Related Work

Prior work addresses three complementary parts of this problem: learning latent predictive representations, preserving control-relevant distinctions under visual shifts, and evaluating learned dynamics.

Latent predictive world models. DreamerV3 and TD-MPC2 illustrate how learned dynamics can support control by predicting the consequences of candidate actions [11, 12]. JEPAs replace pixel reconstruction with prediction in representation space, first for images and then for video [16, 1, 3, 2]. LeWM instantiates this idea as an end-to-end JEPA world model stabilized by SIGReg [19, 7], while PLDM provides a different joint-embedding dynamics design [25, 26, 18]. Analyses of latent prediction explain why it can suppress nuisance or noisy features [28, 17, 14]; Seq-JEPA studies the related balance between invariant and equivariant prediction [9]. These results motivate latent predictive learning, but our focus is checkpoint evaluation: whether the discrepancy between two visual views of the same trajectory contracts or grows as the predictor rolls them forward under shared actions.

Robust visual control and selective invariance. DrQ, DrQ-v2, and SODA improve visual control through training-time augmentation [15, 33, 13]. ViGMO and VIBR learn invariances in latent or value space [20, 6], whereas ReOI modifies observations at test time to reduce distractor sensitivity in visual model-predictive control [5]. These methods seek to train or repair a more robust controller. We instead ask how to audit visual sensitivity in an already trained predictive model.

Control-aware representation learning also explains why robustness cannot mean removing every difference. Bisimulation-based methods preserve distinctions in rewards and transitions [8, 34, 24, 27], while value-equivalent and value-aware models preserve distinctions that matter to downstream value or planning [10, 30]. This motivates the two-sided structure of our diagnostic: same-state visual views should agree after prediction, but low disagreement is considered meaningful only when pairs marked as different by task state coordinates remain separated. Our state-coordinate test is narrower than a learned bisimulation or semantic criterion; its role is to expose the constant-representation failure mode.

Diagnostics of learned dynamics. Recent work tests whether learned dynamics retain action or temporal structure through inverse prediction, latent action decoding, cycle consistency, group-action tests, or compatibility with generated futures [4, 35, 23, 21, 31]. Other methods use action-conditioned consistency as a training objective [32] or diagnose kinematic failures in long-horizon rollouts [22]. These studies motivate testing the predictor directly instead of inferring its behavior from encoder geometry.

ACPC contributes a complementary paired-view test. It holds the action sequence fixed and measures how much a state-preserving visual change alters the predicted trajectory. We then relate this change to prediction-error drift against a common observed future, cost movement within a fixed candidate pool, and adaptive-CEM decision regret. For checkpoint screening, ATR summarizes visual sensitivity and SMPR tests separation on selected task-state coordinates. We calibrate their numerical thresholds separately within each model family.

3 Action-Conditioned Predictive Consistency as a Diagnostic

A visual perturbation can change the pixels without changing the underlying control state. Even so, it can move the encoded history, and the learned dynamics can amplify or contract that change before the planner computes a cost. We study this process at three levels. First, a local geometric analysis compares perturbed versions of one history with other clean histories. Second, ACPC follows a clean–perturbed pair through the complete predicted rollout under the same actions. Third, ATR summarizes ACPC across logged histories, while SMPR checks whether selected pairs with different task-state labels remain separated. ATR and SMPR must be interpreted together: low visual sensitivity is useful only if the representation retains relevant state distinctions.

3.1 Local representation geometry under visual perturbations

Start with one clean history and generate several visually perturbed versions of it. After encoding or rolling forward each version, how far do the perturbed representations spread from the clean representation? In particular, is this spread smaller or larger than the distance to the nearest other clean history? This comparison provides a descriptive view of local representation geometry before we introduce the controlled rollout metric.

Let $\mathcal{V} \in \{\mathrm{enc}, \mathrm{roll}\}$ denote either the encoder output or the representation after eight autoregressive rollout steps. For anchor history i, let $v^{(0)}_{i,\mathcal{V}}$ be its clean representation and $v^{(m)}_{i,\mathcal{V}}$, $m = 1, \ldots, M$, the representations of M perturbed views. In the rollout space, the clean and perturbed views of each anchor use the same recorded action sequence. We define the sampled visual radius, nearest-clean spacing, and their ratio as

$$\begin{aligned} r^{\mathrm{vis}}_{i,\mathcal{V}} &= \max_{1 \le m \le M} \left\| v^{(m)}_{i,\mathcal{V}} - v^{(0)}_{i,\mathcal{V}} \right\|_2, \\ d^{\mathrm{NN}}_{i,\mathcal{V}} &= \min_{j \ne i} \left\| v^{(0)}_{i,\mathcal{V}} - v^{(0)}_{j,\mathcal{V}} \right\|_2, \qquad \rho^{\mathrm{local}}_{i,\mathcal{V}} = \frac{r^{\mathrm{vis}}_{i,\mathcal{V}}}{d^{\mathrm{NN}}_{i,\mathcal{V}}}. \end{aligned} \tag{1}$$

The figures abbreviate $\rho^{\mathrm{local}}_{i,\mathcal{V}}$ as r/NN. When this ratio is below one, every sampled perturbed view of anchor i lies closer to its clean representation than that representation lies to the nearest other clean anchor. A ratio of at least one means that the largest sampled displacement reaches or exceeds this nearest-clean distance. This is a geometric comparison; the nearest clean anchor need not lie across a semantic or task-relevant boundary.

The ratio uses only anchor i's perturbation radius. It does not account for the perturbation radius around the neighboring anchor. We therefore add a symmetric test: the enclosing ball around anchor i is *fully disjoint* if it is separated from the enclosing ball around every other anchor:

$$\left\| v_{i,\mathcal{V}}^{(0)} - v_{j,\mathcal{V}}^{(0)} \right\|_2 > r_{i,\mathcal{V}}^{\text{vis}} + r_{j,\mathcal{V}}^{\text{vis}} \quad \text{for every } j \neq i. \tag{2}$$

Across the sampled anchors, we report three summaries: the median r/NN ratio; the count and fraction satisfying $r_{i,\mathcal{V}}^{\text{vis}} < d_{i,\mathcal{V}}^{\text{NN}}$; and the count and fraction satisfying the fully-disjoint condition in Equation (2). The second is a one-sided nearest-center test, whereas the third is a symmetric certificate for the enclosing balls in the original high-dimensional representation. It is unrelated to whether the qualitative t-SNE display ellipses overlap.

These quantities show local contraction and separation, but they cannot determine whether a perturbation changes a prediction or planning decision. The nearest clean anchor is chosen in learned space and need not represent a task-relevant or semantic difference. Different clean rollout centers also come from their own recorded action sequences, so their spacing can mix state and action effects. The encoder output and final rollout step also omit all intermediate predictions. In addition, the sampled maximum and nearest-clean distance depend on the number of views, the density of clean anchors, and the representation scale. Neither quantity bounds changes in future prediction error or planner cost, and a collapsed representation can make every radius small.

These limitations motivate ACPC, which compares clean and perturbed views of the same history under exactly the same action sequence throughout the predicted rollout. The rollout-space comparison above is its final-step component: when $\alpha_H > 0$, the final-step displacement for any view is at most $\text{ACPC}_H/\sqrt{\alpha_H}$. Local geometry therefore motivates, but does not replace, the complete action-conditioned comparison.

3.2 Paired rollout radius

ACPC asks a direct question: if only the visual appearance of a history changes, how much does the model's predicted future change under a fixed action sequence? Let h be the clean observation–action history consumed by the checkpoint's encoder, and let $\tilde{h}$ be a visually perturbed view of the same underlying state trajectory. A frozen encoder gives $z = E_\theta(h)$ and $\tilde{z} = E_\theta(\tilde{h})$. Both branches receive the same action sequence $\mathbf{a} = (a_0, \ldots, a_{H-1})$. For its length-$k$ prefix, define

$$\hat{z}_k = F_\theta^k(z, \mathbf{a}_{0:k-1}), \qquad \hat{\tilde{z}}_k = F_\theta^k(\tilde{z}, \mathbf{a}_{0:k-1}). \tag{3}$$

Here F_θ is the action-conditioned predictor and H is the number of autoregressive future steps. The planner does not read raw predictor outputs: it scores candidates in a normalized projection of the embedding space. The map Π denotes that projection, so every distance below lives in the same space in which planner costs are computed; our implementation stores these projected embeddings directly, and Π then requires no extra computation. With nonnegative weights $\sum_{k=1}^{H} \alpha_k = 1$, stack the whole rollout as

$$\bar{G}_{\mathbf{a}}(z) = \left[\sqrt{\alpha_1}\Pi(\hat{z}_1)^\top \quad \cdots \quad \sqrt{\alpha_H}\Pi(\hat{z}_H)^\top \right]^\top. \tag{4}$$

Action-Conditioned Predictive Consistency (ACPC) is the distance between the two weighted predicted rollouts:

$$\text{ACPC}_H(h, \tilde{h}, \mathbf{a}) = \left\| \bar{G}_{\mathbf{a}}(E_\theta(h)) - \bar{G}_{\mathbf{a}}(E_\theta(\tilde{h})) \right\|_2 = \left(\sum_{k=1}^{H} \alpha_k \left\| \Pi(\hat{z}_k) - \Pi(\hat{\tilde{z}}_k) \right\|_2^2 \right)^{1/2}. \tag{5}$$

Encoder shift $\|z - \tilde{z}\|_2$ measures the effect of the visual change before prediction. ACPC measures its effect on the predicted trajectory after both histories have been propagated under the same actions. Holding the actions fixed isolates sensitivity to the visual perturbation; it does not establish whether the model responds correctly to other actions. Unless stated otherwise we use uniform weights $\alpha_k = 1/H$ and the fixed protocol horizon $H = 8$, the longest horizon for which every logged evaluation window contains a complete observed future (Appendix B). The planner analyses in Section 4.5 instead use $H = 5$, matching the five-action planning horizon of the evaluated CEM configuration.

3.3 Common-future error drift

ACPC can be computed without observing the true future. When a logged future is available, however, the same quantity bounds how much the visual perturbation can change prediction error. Let $Y_{\mathbf{a}}^H$ be one actually observed future under the same recorded action sequence, projected, stacked, and weighted exactly as in Equation (4). This target is not produced by either prediction branch and is used only for evaluation. Comparing both branches with this one future isolates the error change caused by the visual perturbation.

Proposition 1 (Common-future error-drift bound)**.** *Define*

$$e_h = \|\bar{G}_{\mathbf{a}}(E_\theta(h)) - Y_{\mathbf{a}}^H\|_2, \qquad e_{\tilde{h}} = \|\bar{G}_{\mathbf{a}}(E_\theta(\tilde{h})) - Y_{\mathbf{a}}^H\|_2.$$

Then, for every paired sample,

$$|e_{\tilde{h}} - e_h| \leq \mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}). \tag{6}$$

The adverse increase $(e_{\tilde{h}} - e_h)_+$*, where* $(x)_+ = \max(x, 0)$*, obeys the same bound.*

Proof. Apply the reverse triangle inequality in the weighted projected-rollout space. Both errors must use the same $Y_{\mathbf{a}}^H$. □

The proposition bounds the difference between two errors, not either prediction's absolute error. Both predictions may be inaccurate even when ACPC is small. The result states only that their errors relative to the same future cannot differ by more than ACPC. ACPC itself still does not require the realized future. The prediction experiment in Section 4.4 uses that future only to evaluate whether ACPC is informative about the bounded quantity, which we call the *error drift* $d = |e_{\tilde{h}} - e_h|$.

3.4 Candidate-cost drift and planner stability

We next connect ACPC to planning. A visual perturbation moves each candidate's predicted endpoint and therefore may change its cost. If no candidate can move far enough to cross the clean cost gap, the selected candidate remains unchanged. This is the radius–margin argument formalized below. For candidate action sequence $\mathbf{a}^j$, let

$$x_j = \Pi(F_\theta^H(E_\theta(h), \mathbf{a}^j)), \qquad \tilde{x}_j = \Pi(F_\theta^H(E_\theta(\tilde{h}), \mathbf{a}^j))$$

be the final clean and perturbed predicted representations. Their displacement $r_j = \|x_j - \tilde{x}_j\|_2$ is the final-step component of the candidate-specific $\mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)$. If $\alpha_H > 0$, then $r_j \leq \mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)/\sqrt{\alpha_H}$; the planner experiments in Section 4.5 record r_j directly.

Proposition 2 (Squared goal-cost drift and fixed-pool stability)**.** *Suppose the planner scores candidate j against fixed goal embedding g by $C_j = \|x_j - g\|_2^2$ and $\tilde{C}_j = \|\tilde{x}_j - g\|_2^2$. Define*

$$b_j = r_j\big(\|x_j - g\|_2 + \|\tilde{x}_j - g\|_2\big). \tag{7}$$

Then every candidate satisfies $|\tilde{C}_j - C_j| \leq b_j$.

Let w and $\tilde{w}$ be the clean and perturbed winners of the same pool. The clean-history decision regret obeys

$$0 \leq C_{\tilde{w}} - C_w \leq b_{\tilde{w}} + b_w. \tag{8}$$

If the clean winner is unique and

$$\min_{j \neq w}\big(C_j - C_w - b_j - b_w\big) > 0, \tag{9}$$

then the perturbed branch has the same winner. If $\mathcal{E}$ is the clean top-k elite set and

$$\min_{i \in \mathcal{E},\, j \notin \mathcal{E}}\big(C_j - C_i - b_j - b_i\big) > 0, \tag{10}$$

then the perturbed branch has exactly the same elite set.

The quantity b_j is a candidate-specific upper bound on the movement of the squared goal cost. The regret inequality bounds the extra clean-history model cost of choosing the perturbed winner. The last two conditions subtract the allowed movements from each clean candidate gap; a positive remainder means that the gap cannot be crossed. These statements use the evaluated endpoints, not a global Lipschitz approximation. Proofs and equality cases are in Appendix A. Because every candidate must be evaluated under both histories, these certificates support a post-hoc audit rather than a cheap online pre-screen.

CEM repeatedly fits its next proposal to the current elite set. With common random numbers, identical proposals generate the same ordered candidate pool, and an identical certified elite set produces the same next proposal.

Corollary 1 (Conditional CEM stability)**.** *Consider clean and perturbed CEM runs initialized by the same proposal and sampled with common random numbers. While their ordered candidate pools remain aligned, if* Equation (10) *holds at the current iteration, both runs fit the same next proposal. If it holds at every iteration, their pools, proposals, and final actions remain aligned by induction.*

The guarantee stops at the first iteration for which the elite condition fails. It concerns candidate selection and model cost under paired planner evaluations; it is not a statement about simulator return. Appendix G specifies the fixed-pool and adaptive-CEM evaluation contract.

3.5 Checkpoint-level radius and proxy separation

ACPC measures one clean–perturbed history pair. To evaluate a checkpoint, we must summarize many such measurements. We must also distinguish genuine visual stability from representation collapse: a constant representation makes every ACPC value zero but removes all state distinctions. We therefore use ATR to summarize sensitivity across histories and SMPR to test whether selected pairs with different task-state labels remain separated.

Evaluation uses a fixed set of *anchors*. Each anchor consists of one logged history window, its H-step observed future, and its recorded actions. We first normalize ACPC separately for each anchor and then aggregate across anchors. Throughout this section, Q_q denotes the empirical q-quantile, n is the number of anchors, and M is the number of perturbation draws per anchor. For anchor i, let $u_{i,0}^{\text{obs}}$ be the embedding of the last clean history observation in the projected space selected by Π, and let $u_{i,1:H}^{\text{obs}}$ be the embeddings of the next H actually observed clean frames in that same space. Define the clean-motion scale

$$s_i = Q_{0.50}\Big(\big\{\|u_{i,k}^{\text{obs}} - u_{i,k-1}^{\text{obs}}\|_2\big\}_{k=1}^{H}\Big). \tag{11}$$

This anchor-specific scale expresses rollout distances in units of the anchor's ordinary clean motion; it uses no task outcome. With a small numerical stabilizer $\varepsilon_{\text{norm}}$ (value in Appendix B) and perturbation draw m, define

$$R_i^{(m)} = \frac{\text{ACPC}_H(h_i, \tilde{h}_i^{(m)}, \mathbf{a}_i)}{s_i + \varepsilon_{\text{norm}}}, \qquad \bar{R}_i = \frac{1}{M}\sum_{m=1}^{M} R_i^{(m)}. \tag{12}$$

The raw *Action-Conditioned Trajectory Radius* (ATR) is

$$\text{ATR}_q^{\text{raw}}(\theta) = Q_q\big(\{\bar{R}_i\}_{i=1}^{n}\big). \tag{13}$$

ATR is one number per checkpoint. It reports a high quantile of normalized ACPC across the sampled histories and therefore emphasizes histories with relatively high visual sensitivity. We use q90 because a mean can hide a small set of high-radius anchors; checkpoint-level conclusions are stable across horizons $H \in \{1, 2, 4, 8\}$ and quantiles q80–q95 (Table 4). This is a descriptive summary, not a candidate-distribution tail guarantee.

ATR alone cannot rule out a collapsed representation. SMPR provides the complementary separation test. Let y_i be the standardized logged task-state vector at the end of anchor i's observed H-step future, and let $\ell(y_i)$ be a declared task-specific coordinate label. Let $d_{0.35}$ be the q35 of all off-diagonal Euclidean distances among the standardized endpoint states; this neighborhood radius and the margin below are protocol constants fixed a priori. Each eligible anchor chooses the nearest neighbor $j(i)$ with a different label and distance at most $d_{0.35}$. The predictor rolls both clean histories under the anchor actions $\mathbf{a}_i$; for the neighbor these are not the actions it actually executed, but reusing the anchor's actions keeps a single shared action sequence on both sides. Their normalized proxy-different distance is

$$D_i^{\text{diff}} = \frac{\big\|\bar{G}_{\mathbf{a}_i}(E_\theta(h_i)) - \bar{G}_{\mathbf{a}_i}(E_\theta(h_{j(i)}))\big\|_2}{s_i + \varepsilon_{\text{norm}}}. \tag{14}$$

Both histories use the same rollout metric, the same action sequence $\mathbf{a}_i$, and the same normalization s_i. This ensures that visual sensitivity and state separation are measured on a comparable scale. The *Selective Margin Pass Rate* (SMPR) is

$$\text{SMPR}_{q,\delta}(\theta) = \frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} \mathbf{1}\big[D_i^{\text{diff}} > \text{ATR}_q^{\text{raw}}(\theta) + \delta\big], \tag{15}$$

where $\mathcal{I}$ contains anchors with an eligible label-crossing neighbor. The reported protocol fixes $q = 0.90$ and normalized margin $\delta = 0.10$. SMPR is the fraction of tested pairs with different proxy labels whose rollout distance exceeds the same-state radius by more than this margin.

The local ball analysis and SMPR both compare a radius with a margin, but they answer different questions. The local analysis uses distances in learned space to describe the sampled neighborhood around each clean anchor. It examines either the encoder output or the final rollout step and applies a symmetric two-ball test. SMPR instead selects nearby histories using a declared task-state label. It rolls both histories under the anchor's actions, compares their complete weighted trajectories, and asks whether their distance exceeds the checkpoint-level q90 same-state radius. Thus, the local analysis is a geometric case study, whereas ATR and SMPR form the checkpoint-level diagnostic.

ATR and SMPR must be read together. A constant representation makes both ATR and every D_i^{diff} zero, so it fails the SMPR test. Conversely, a model can separate the tested proxy classes while remaining visually sensitive, which yields a large ATR. These proxies test only the declared coordinate distinctions; they do not prove semantic validity, action relevance, or correct behavior. Appendix B gives the exact coordinate rules and eligible-pair counts.

3.6 Checkpoint-level calibration

The raw ATR in Equation (13) is the same-state reference used by SMPR. Raw ATR scales differ across tasks, so comparisons between augmentation levels use a relative value. Let θ_0 be the unaugmented checkpoint for the same task, training run, and model family. Its raw ATR is strictly positive in every evaluated setting, so the following ratio is well defined:

$$\widetilde{\mathrm{ATR}}_q(\theta;\theta_0) = \frac{\mathrm{ATR}_q^{\mathrm{raw}}(\theta)}{\mathrm{ATR}_q^{\mathrm{raw}}(\theta_0)}. \tag{16}$$

Raw ATR remains in the SMPR definition (Equation (15)). Relative ATR equals one for the unaugmented reference and is used only to compare and calibrate checkpoints.

We combine relative ATR and SMPR only after calibrating one threshold for each. For model family $\mathcal{F}$ and visual shift v (Gaussian noise, blur, or resize), let these thresholds be t_R and t_M. Every quantity in the score is measured under the same shift. With a small constant $\varepsilon_{\mathrm{score}}$ (value in Appendix B), define

$$S_{\mathcal{F},v}(\theta;\theta_0) = \min\left\{\frac{t_R - \widetilde{\mathrm{ATR}}_q(\theta;\theta_0)}{|t_R| + \varepsilon_{\mathrm{score}}}, \frac{\mathrm{SMPR}_{q,\delta}(\theta) - t_M}{|t_M| + \varepsilon_{\mathrm{score}}}\right\}. \tag{17}$$

The score satisfies $S \geq 0$ exactly when both conditions pass: relative ATR is at most t_R, and SMPR is at least t_M. No task outcome is needed to score a checkpoint once the thresholds are fixed. Planning success rates are used only to select thresholds on source tasks and evaluate them on other tasks. We calibrate thresholds separately for each model family; the equations do not imply that architectures share numerical thresholds.

For a comparison with reference checkpoint θ_0, define

$$\Delta S_{\mathcal{F},v}(\theta_1;\theta_0) = S_{\mathcal{F},v}(\theta_1;\theta_0) - S_{\mathcal{F},v}(\theta_0;\theta_0). \tag{18}$$

A positive ΔS means only that θ_1 improves on its reference under the same calibrated scoring map. It does not assign an absolute robustness label to either checkpoint.

Table 1 summarizes the chain. The prediction experiment (Section 4.4) evaluates whether ACPC is informative about the error drift bounded by Proposition 1; the CEM experiment (Section 4.5) analyzes the candidate-cost and selection quantities in Proposition 2 and corollary 1; and the cross-task, PLDM, and blur/resize analyses (Sections 4.6 to 4.8) evaluate the calibrated ATR–SMPR score. These are related but distinct empirical claims.

Table 1: Reader guide to the analysis and diagnostic quantities.

Object	Plain-language meaning	Empirical role
Local r/NN and ball separation	Size of a visual perturbation cloud relative to nearby clean representations.	PushT geometry case study
ACPC	Distance between the clean and perturbed predicted futures of one history under shared actions.	Prediction and planner analyses
b_j	Upper bound on how far candidate j's squared goal cost can move.	Fixed-pool planner audit
$\mathrm{ATR}^{\mathrm{raw}}$	q90 radius of the normalized same-state rollout tube.	Tube used by SMPR
D_i^{diff}	Rollout distance to a nearby state with a different coordinate-proxy label.	Proxy-separation test
SMPR	Fraction of proxy-different pairs that stay outside the tube by the margin.	Collapse guard
$\widetilde{\mathrm{ATR}}$, S	Task-relative radius; the smaller (worse) of the two calibrated margins.	Sweep and cross-task screening
ΔS	Change in the calibrated score relative to a reference checkpoint.	Blur/resize ordering

4 Experiments

We first state the common evaluation protocol. We then begin with a descriptive local-geometry case study that motivates the controlled ACPC comparison, before evaluating checkpoint-level behavior, prediction-error drift, planning consequences, and transfer.

4.1 Evaluation setup

Models, tasks, and planning evaluation. LeWM [19] is the primary model family, and PLDM [25, 26] tests whether the diagnostic applies to a second joint-embedding dynamics architecture. We evaluate four control tasks: TwoRoom navigation, PushT planar manipulation, Reacher arm control, and OGBench-Cube (Cube) 3D manipulation. Following the standard LeWM latent-space MPC evaluation, CEM uses the frozen world model to optimize candidate sequences of five actions (plan horizon 5). We report *planning success rate*, the percentage of evaluation episodes that reach the task goal.

We use *visual perturbation* for one sampled change to a history and *visual shift* for the distribution from which that change is drawn. The main evaluation adds Gaussian noise with $\sigma = 0.08$ to the observations while keeping the goal image clean. Additional evaluations use Gaussian blur ($k = 15$) and resize (scale 0.25).

For each model family and task, the Gaussian-augmentation sweep contains nine checkpoints: one *unaugmented checkpoint*, trained without noise augmentation, and eight checkpoints trained with full-sequence noise $\sigma_{\max} \in \{0.01, \ldots, 0.08\}$. Each LeWM task–augmentation setting has three training runs (seeds 3072/3073/3074). Each run is evaluated with seeds 42/43/44 and 100 episodes per evaluation seed. We use *task–run cell* for one task and one training run.

The training run is the unit of replication. The three evaluation seeds measure only within-run variability, so the reported means and dispersions across runs are descriptive rather than population estimates. The PLDM sweep has one run for each of its 36 task–augmentation settings, with the same evaluation seeds and episode count.

Diagnostic measurements. ATR uses 100 logged anchors, five perturbation draws, eight predicted steps, uniform horizon weights, within-anchor draw averaging, and a q90 summary across anchors. SMPR selects pairs within a global q35 radius among standardized endpoint states and tests their separation from the same-state q90 reference with margin 0.10. Appendix B gives the complete normalization and threshold protocol. The raw ATR defines the tube in SMPR; checkpoint thresholding uses the task-relative ATR in Equation (16).

Threshold evaluation. A Gaussian-sweep checkpoint meets the *success-rate criterion* if it recovers at least 80% of the improvement from the unaugmented checkpoint to the best checkpoint at evaluation noise $\sigma = 0.08$, while losing no more than five percentage points on clean observations. Both constants were fixed a priori. We select (t_R, t_M) on every nonempty proper subset of the four tasks and apply the thresholds unchanged to the remaining tasks. The one-, two-, and three-source-task settings produce $4 + 6 + 4 = 14$ directional source/evaluation partitions. Metrics first average training runs within each evaluation task and then weight tasks equally; checkpoint rows are never treated as independent replicates. Success rates are used to select and evaluate thresholds, but they are not inputs to ACPC, ATR, or SMPR.

For blur and resize, each task uses the Gaussian thresholds selected on the other three tasks. We compare 24 checkpoint pairs (four tasks $\times$ three runs $\times$ two shifts). Each pair contains the unaugmented checkpoint and the checkpoint trained at $\sigma_{\max} = 0.08$. A pair is positive when success under the shift improves by at least five percentage points and clean success decreases by no more than five points. We make no blur- or resize-specific adjustment. This analysis also does not benchmark alternative signals such as encoder shift or one-step ACPC.

4.2 Motivating case study: local geometry before and after prediction

We begin with the descriptive geometric audit introduced in Section 3.1. Its purpose is to visualize the phenomenon that motivates ACPC, not to establish task-level robustness. Using a fixed PushT case study, we ask whether augmentation makes perturbed views of the same history more compact relative to other clean histories. We compare LeWM training run 3072 before and after full-sequence Gaussian augmentation with $\sigma_{\max} = 0.08$. For each of 128 logged histories, we generate one clean view and 18 perturbed views—six draws at each probe standard deviation in $\{0.01, 0.04, 0.08\}$. We compare the encoder output with the representation after eight autoregressive rollout steps. All ratios and counts use the original 192-dimensional projected latent space in which the planner computes costs. The t-SNE visualization [29] reproduces the deterministic per-panel projections used in the original Appendix visualization and is qualitative only.

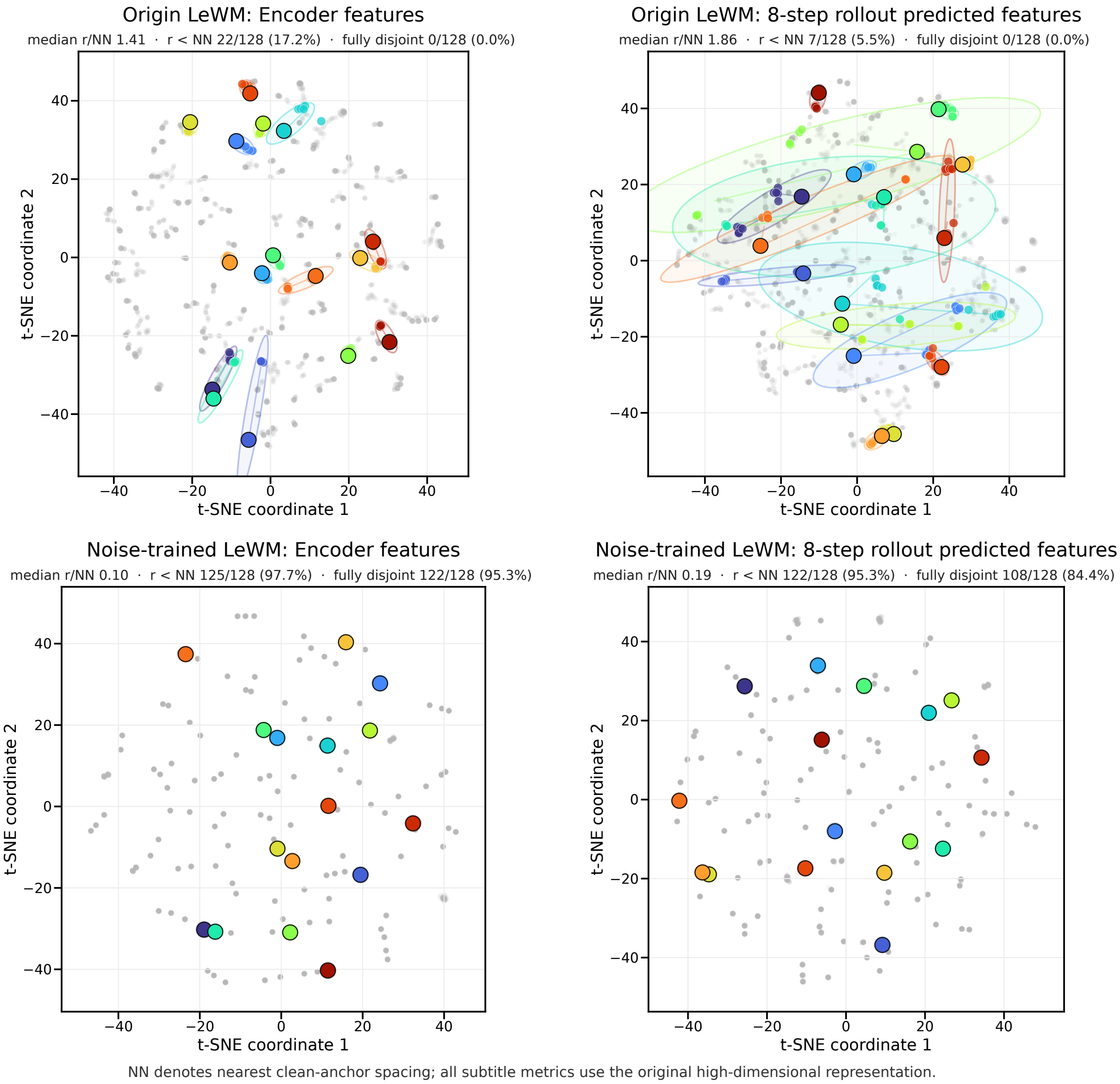

Figure 1: Descriptive local-geometry case study motivating the controlled ACPC comparison. The fixed PushT case study uses LeWM training run 3072. Rows compare the unaugmented and $\sigma_{\max} = 0.08$ Gaussian-augmented checkpoints; columns show the encoder output and the representation after eight autoregressive rollout steps. Each color denotes one of 16 highlighted histories: the large black-rimmed marker is its clean anchor, smaller same-color markers are its 18 perturbed views, and the faint ellipse is their 90% t-SNE envelope; gray points show all other histories and views. Under strong contraction, smaller markers and ellipses can lie beneath the clean marker. Each subtitle reports, across all 128 anchors, the median r/NN, the count and fraction with $r <$ NN, and the count and fraction whose high-dimensional enclosing balls are fully disjoint. The t-SNE coordinates and ellipses are qualitative and enter none of these metrics.

Without augmentation, the median r/NN is 1.41 at the encoder and 1.86 after eight rollout steps, and none of the 128 anchor clouds is fully disjoint. After Gaussian augmentation, the corresponding ratios fall to 0.10 and 0.19; the fully disjoint fractions rise to 95.3% at the encoder and 84.4% after the rollout. Thus, in this fixed case study, augmentation makes perturbed views of the same history substantially more compact relative to nearby clean histories. Much of this separation remains after prediction.

This case study shows that augmentation can contract perturbation-induced variation relative to nearby clean histories, including after prediction. It does not establish task-semantic separation or planning robustness. We therefore turn to the checkpoint-level ATR–SMPR diagnostic and its relationship with planning performance across the complete augmentation sweep (Figure 2).

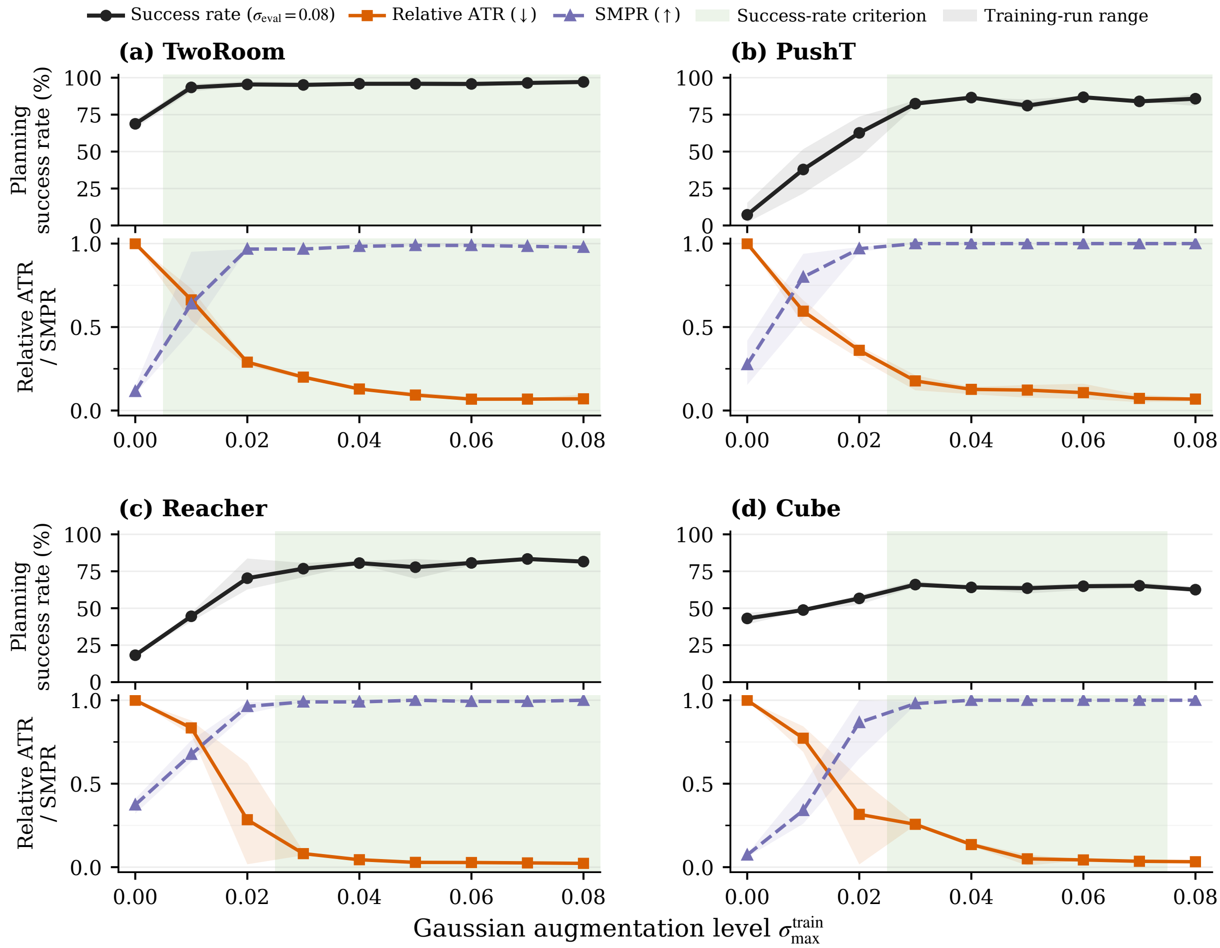

Figure 2: Checkpoint-level planning performance and diagnostics across the complete Gaussian-augmentation sweep. We report planning success rate at evaluation noise $\sigma = 0.08$, relative ATR, and SMPR. Lines are across-run means; shaded ranges span the three run means; green bands mark levels meeting the success-rate criterion (Section 4.1) in a majority of runs. Lower ATR (relative to the unaugmented checkpoint) and higher SMPR are favorable.

4.3 Planning performance under observation noise

Do ATR and SMPR track the recovery of planning performance across the full Gaussian-augmentation sweep? At evaluation noise $\sigma = 0.08$, the unaugmented LeWM checkpoints lose 25.9 ± 2.4, 74.6 ± 5.6, 41.0 ± 1.4, and 22.1 ± 2.8 percentage points in success rate on TwoRoom, PushT, Reacher, and Cube, respectively. Gaussian augmentation recovers performance over a range of training levels whose location differs by task. Figure 2 compares success rate with ATR and SMPR across the complete sweep.

Every augmentation level that meets the success-rate criterion has lower ATR and higher SMPR than the corresponding unaugmented checkpoint. Neither diagnostic, however, changes monotonically at every augmentation level. **Takeaway:** recovered planning performance coincides with lower visual sensitivity and greater separation of the tested state pairs. This motivates combining ATR and SMPR. The next two experiments separately test whether ACPC uses action information and whether it tracks planning-relevant quantities (Sections 4.4 and 4.5).

4.4 Predicting the perturbation-induced error drift

Does ACPC help predict how much a visual perturbation changes multi-step prediction error? For each history pair, Proposition 1 shows that the two errors against the same logged future can differ by at most ACPC_H. We test whether measured ACPC is informative about this bounded quantity, the error drift $d = |e_{\tilde{h}} - e_h|$. We evaluate at the protocol horizon $H = 8$, the longest horizon with a complete logged common future for every anchor; Table 4 shows that the checkpoint-level conclusions are stable for $H \in \{1, 2, 4, 8\}$.

Data and protocol. The analysis uses the unaugmented checkpoint of each task and training run. Trajectories are partitioned into 16 disjoint groups. Each history pair contributes rows at Gaussian probe severities $\{0.02, 0.05, 0.08\}$ with two perturbation draws; zero-severity probes serve only as identity checks. A ridge model is fitted on 15 groups and evaluated on the remaining group, rotating through all 16, and all rows from one trajectory group stay together. We report the mean absolute error (MAE) on groups excluded from model fitting.

Three nested regressions. Every model contains the probe severity and the encoder-history distance $\|z - \tilde{z}\|_2$.

- **Base** adds one-step ACPC under the recorded action: the information available without a multi-step rollout.
- **Base+Control$_8$** adds the strongest *destroyed-action control*: eight-step ACPC recomputed with *zeroed* actions (every action set to zero), *swapped* actions (the recorded actions of a different trajectory), or *shuffled* actions (the anchor's own actions in permuted temporal order). Each control performs the same eight-step rollout, so it carries rollout length without the recorded action sequence. "Strongest" is an oracle choice: in each task–run cell we report whichever of the three controls attains the lowest test MAE, which makes the comparison conservative.
- **Base+ACPC$_8$** instead adds eight-step ACPC under the recorded actions—the only feature whose action sequence is the one that generated the logged future, and hence the feature matching the right-hand side of Proposition 1.

All eight-step features use uniform weights $\alpha_k = 1/8$ in Equation (5). If Base+ACPC$_8$ improves on Base, multi-step information helps; if it also improves on Base+Control$_8$, the gain is attributable to the recorded actions rather than to merely rolling out eight steps.

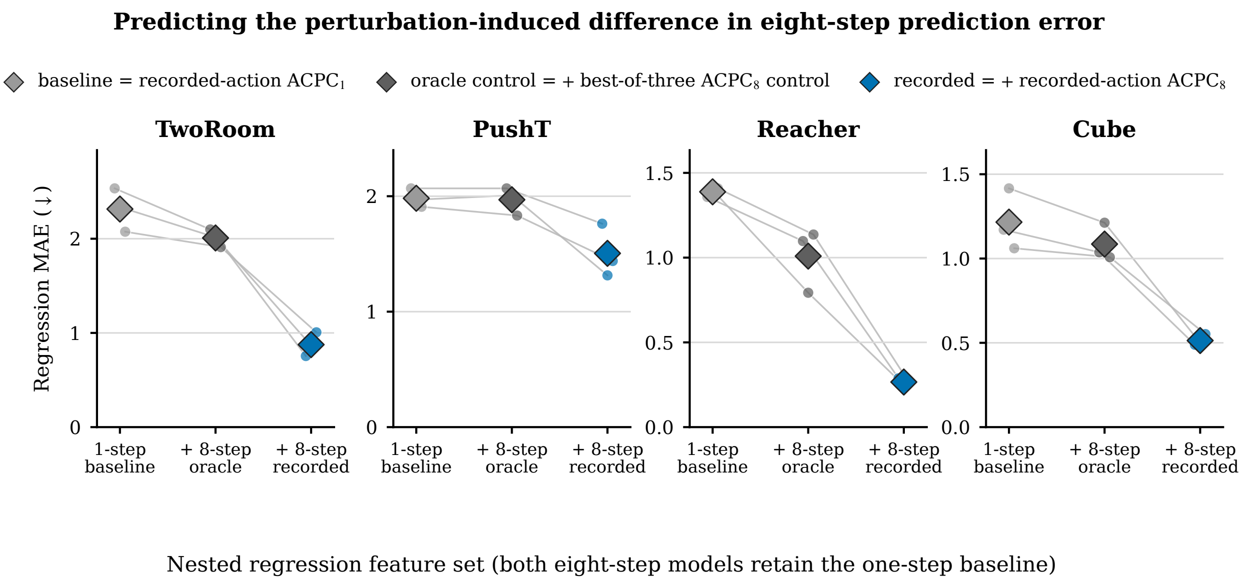

Figure 3: Cross-validated MAE for predicting the error drift d on the unaugmented checkpoints; lower is better. Each model is evaluated on trajectory groups excluded from fitting. Small points are training runs, thin lines join the same run, and diamonds are task means. The in-figure legend labels correspond to Base, Base+Control$_8$, and Base+ACPC$_8$ as defined in Section 4.4. Base+ACPC$_8$ is lowest in all 12 task–run cells; latent-error scales are task-specific, so compare within a panel.

Results. Base+ACPC$_8$ attains the lowest cross-validated MAE in all 12 task–run cells (Figure 3). For each training run we average the four task-level relative MAE reductions equally and report the mean $\pm$ sample standard deviation across the three training runs: the reduction is $55.9 \pm 4.7\%$ relative to Base and $51.3 \pm 3.5\%$ relative to Base+Control$_8$. These results concern prediction of the error drift; they do not compare the absolute forecast accuracy of one-step and eight-step world models. Appendix C reports the task-level values and selected controls. **Takeaway:** eight-step ACPC carries information about the bounded error change that neither encoder shift, one-step ACPC, nor any same-horizon destroyed-action control provides.

4.5 Relating ACPC to CEM cost and decision changes

We next ask whether ACPC indicates how much a visual perturbation changes CEM candidate costs and decisions. We answer in two steps. First, we verify the deterministic bounds in Proposition 2 using paired clean and perturbed candidate pools. Second, we test whether ACPC improves the prediction of observed cost and decision changes beyond simple baseline features. Both analyses use identical candidate pools and compare the unaugmented checkpoint with the checkpoint trained at $\sigma_{\max} = 0.08$.

The broad analysis uses 100 histories per task and training run with a reduced CEM budget. A separate confirmation uses the same fixed set of 16 histories per task with the deployed CEM budget. For each candidate, we compute five-step ACPC under that candidate's own action sequence, matching the evaluated planning horizon. Appendix G reports the complete candidate, elite, and iteration counts.

Verifying the deterministic bounds. The reduced-budget analysis contains 9,600 records (100 histories $\times$ four tasks $\times$ three runs $\times$ two training conditions $\times$ four perturbation severities). The candidate-cost bound has no numerical violations. Here, the *winner* is the lowest-cost candidate, and the *elite set* contains the candidates used for the next CEM update. The winner and elite-set certificates never predict stability when the corresponding selection changes. Identity probes, which duplicate the clean history at zero perturbation severity, give exactly zero five-step ACPC and zero RMS change in the first planned action. Whenever the elite-set certificate holds, the clean and perturbed CEM updates also remain aligned, as Corollary 1 requires. These checks validate the paired model-cost calculation; they are not guarantees about environment return.

Predicting cost and decision changes. We next ask whether five-step ACPC adds information beyond a baseline that already contains perturbation severity, training condition, one-step candidate-specific ACPC, and the clean cost gap between the best and second-best candidates. The three prediction targets are:

- the *largest candidate-cost change*: the maximum absolute cost change among candidates shared by the clean and perturbed pools;
- *decision regret*: the additional clean-history model cost of the plan selected from the perturbed history, relative to the plan selected from the clean history; and
- the *first-action change*: the root-mean-square (RMS) difference between the first actions of the final clean and perturbed plans—the actions that MPC would execute.

Decision regret uses the model's squared latent goal cost, not simulator return. For each training run, we fit a ridge model on three tasks and evaluate it on the remaining task, repeating the procedure for all four choices of evaluation task. Table 2 reports how much adding five-step ACPC reduces prediction MAE; Figure 4 shows the paired errors.

Table 2: Reduction in prediction MAE after adding candidate-specific five-step ACPC. Models are fitted on three tasks and evaluated on the remaining task; higher is better.

Prediction target	MAE reduction (%) ↑ mean $\pm$ run-to-run std. dev.	Task–run evaluations improved (/12)
Largest candidate-cost change in the shared pool	7.9 $\pm$ 1.4	8/12
Adaptive-CEM decision regret	15.2 $\pm$ 2.0	12/12
RMS change in the first planned action	1.1 $\pm$ 0.5	10/12

MAE is computed on the $\log(1 + \text{target})$ scale. The four evaluation tasks are weighted equally within each training run; entries then report the mean and standard deviation across three runs. The final column counts the 12 task–run evaluations separately.

Five-step ACPC gives the clearest improvement for adaptive-CEM decision regret: prediction MAE decreases by $15.2 \pm 2.0\%$ (mean $\pm$ standard deviation across training runs), with an improvement in all 12 task–run evaluations. For the largest candidate-cost change, MAE decreases by $7.9 \pm 1.4\%$, with improvements in 8 of 12 task–run evaluations and in the task-level mean for three of four tasks. The $1.1 \pm 0.5\%$ reduction for the first-action change is negligible and does not support an incremental claim. In all three analyses, five-step ACPC has a higher Spearman rank correlation with the target than one-step ACPC in every task–run evaluation.

Effect of augmentation on CEM decisions. At perturbation severity 0.08, the task-level mean rate of retaining the best fixed-pool candidate is 0.92–0.96 after Gaussian-augmented training, compared with 0.02–0.13 for the unaugmented checkpoints. Mean positive decision regret is also lower with augmentation in every task–run cell: 3.64 ± 0.68 versus 227.04 ± 16.92 at reduced budget and 2.65 ± 1.43 versus 244.89 ± 27.08 at full budget. Each value first averages histories and the four tasks equally within a training run, then reports the mean

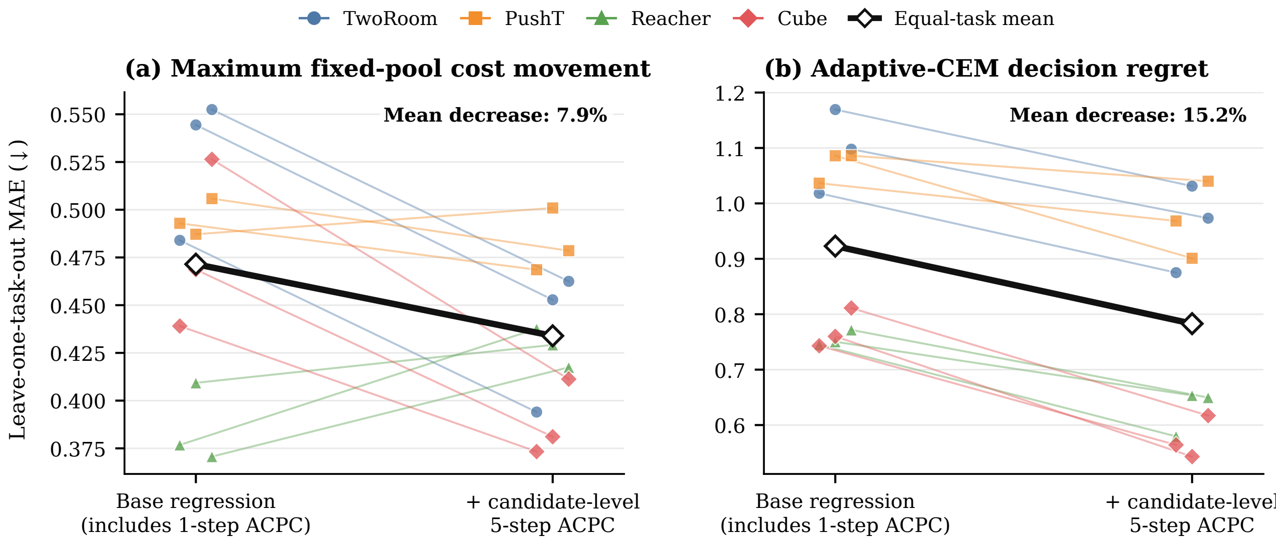

Figure 4: Prediction MAE before and after adding candidate-specific five-step ACPC; lower is better. For each point pair, the model is fitted on three tasks and evaluated on the remaining task for one training run; thin lines connect the two errors from that same evaluation. The thick black pair averages the four evaluation tasks equally within each run and then averages the three runs. Annotations give the corresponding relative MAE reduction. Errors are computed on the $\log(1 + \text{target})$ scale. The first-action target is not shown because its improvement is negligible.

and standard deviation across the three runs. Regret is measured in the model's squared latent goal cost, not simulator return, and latent-cost scales differ by task, so these contrasts are descriptive; Appendix G gives the task-level values. **Takeaway:** the deterministic certificates hold exactly, and candidate-specific ACPC adds cross-task predictive information about the model-cost consequences of a visual perturbation, most clearly for decision regret.

4.6 Cross-task threshold transfer

Can thresholds calibrated on some tasks screen checkpoints from unseen tasks? We treat the calibration tasks as *source tasks* and the held-out tasks as *evaluation tasks*. Thresholds are selected on every nonempty proper subset of the four tasks and applied unchanged to the remaining tasks. This produces the 14 directional partitions described in Section 4.1 (Figure 5). A checkpoint is positive if it meets the success-rate criterion. We also report *onset error*: the difference, in augmentation grid steps, between the first level accepted by the diagnostic and the first level that meets the criterion.

Balanced accuracy is 0.855, 0.900, and 0.900 when thresholds are selected from one, two, and three source tasks, respectively, with precision/recall of 0.910/0.854, 0.913/0.953, and 0.913/0.953. Mean absolute onset errors are 1.1, 0.5, and 0.5 grid steps; the largest individual partition–task–run errors are 5, 1, and 1 steps. With one source task, balanced accuracy ranges from 0.723 to 0.913 across the four possible sources—the weakest single source is Reacher (Table 9)—so calibration depends on task composition, while two and three source tasks perform similarly on average in this four-task study. **Takeaway:** the threshold-selection *procedure* transfers across these tasks; a single universal threshold value does not. Appendix D reports every partition.

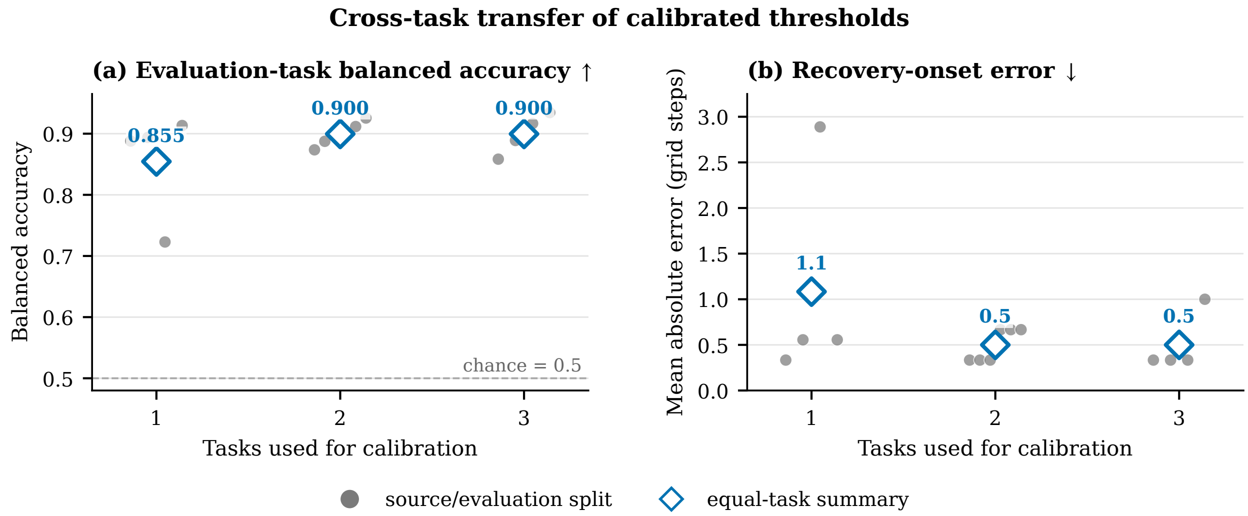

Figure 5: Cross-task transfer of ATR–SMPR thresholds over all 14 source/evaluation partitions; a checkpoint passes when relative ATR $\leq t_R$ and SMPR $\geq t_M$. (a) Balanced accuracy on checkpoints from evaluation tasks (chance is 0.5). (b) Absolute onset error in Gaussian-augmentation grid steps (one step is 0.01 in $\sigma_{\max}$). Gray points are individual partitions; diamonds are equal-task summaries per number of source tasks.

4.7 Application to PLDM

We next ask whether the same diagnostic procedure applies to a different joint-embedding dynamics architecture. We use the complete PLDM Gaussian sweep with the same paired histories, eight-step rollouts, ATR normalization, SMPR guard, success-rate criterion, and threshold-selection procedure. The analysis uses PLDM's encoder and predictor and does not depend on LeWM layers or training losses. Each setting has one training run, so we cannot estimate run-to-run variability. For each PLDM evaluation task, we select thresholds on the other three PLDM tasks and then fix them for its nine checkpoints.

Pooling all 36 task–checkpoint evaluations (four evaluation tasks $\times$ nine checkpoints) gives 0.836 balanced accuracy, 0.789 precision, and 0.882 recall. Task-level balanced accuracies are 0.938, 0.750, 0.500, and 0.875 for TwoRoom, PushT, Reacher, and Cube: on Reacher the transferred thresholds perform at chance. After normalizing ATR within each PLDM task, applying the LeWM thresholds selected on the corresponding three source tasks gives the same aggregate values (0.836/0.789/0.882). **Takeaway:** the equations and evaluation procedure carry over to a second joint-embedding architecture, but numerical calibration remains model-family specific, and the aggregate agreement above does not make thresholds architecture-invariant.

Table 3: The same ACPC analysis applied to the complete PLDM sweep (four tasks $\times$ nine augmentation levels, one training run each; 36 held-out decisions). Row 1 selects thresholds on the other three PLDM tasks; row 2 applies the corresponding LeWM thresholds after within-task ATR normalization. "False pass"/"false miss" count criterion-negative checkpoints accepted and criterion-positive checkpoints rejected.

Calibration	BA	Precision	Recall	False pass	False miss
PLDM thresholds, other three tasks	0.836	0.789	0.882	4	2
LeWM thresholds, PLDM-normalized	0.836	0.789	0.882	4	2

4.8 Transfer to blur and resize

Finally, we ask whether a score calibrated with Gaussian noise can rank checkpoint pairs under previously unseen visual shifts. We test blur ($k = 15$) and resize (scale 0.25), each at one fixed severity. For each task, we select thresholds on the other three Gaussian-noise tasks and apply them to the 24 checkpoint pairs in Section 4.1. No blur or resize result is used to select or adjust a threshold.

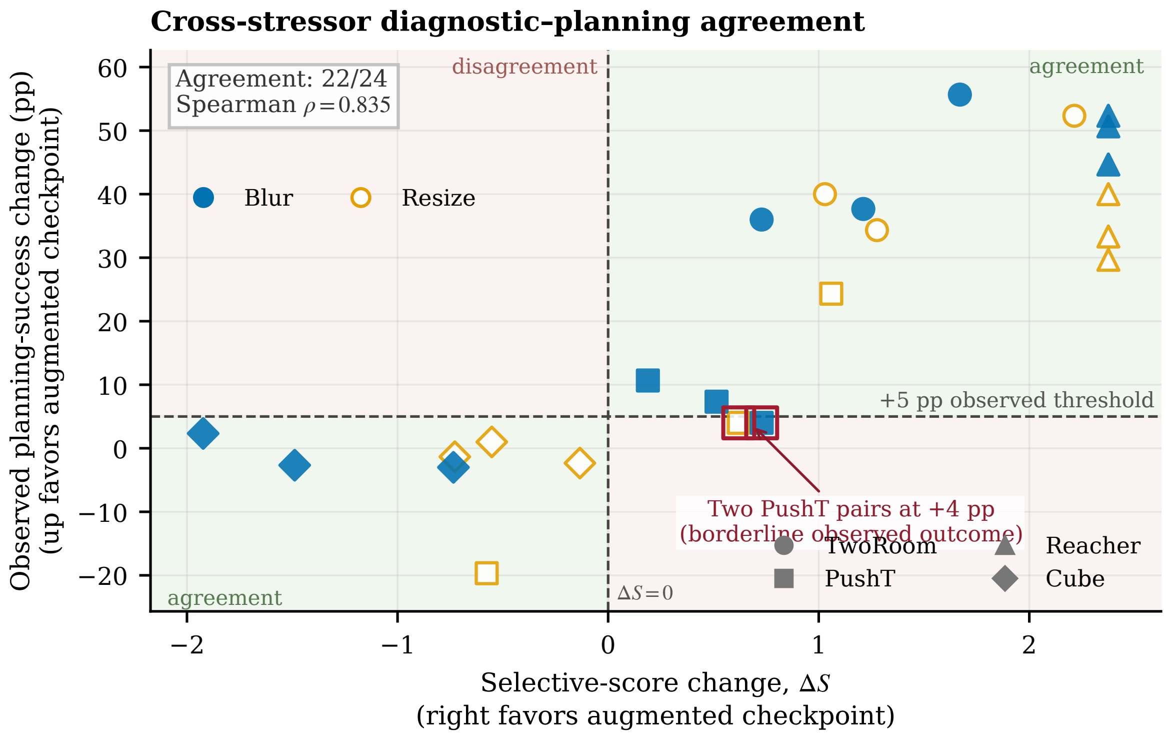

Figure 6: Gaussian-calibrated selective-score change ΔS versus the observed change in planning success under blur and resize. Each of the 24 points pairs an unaugmented checkpoint with its $\sigma_{\max} = 0.08$ counterpart; marker shape denotes task, color/fill denotes the shift. The diagnostic predicts improvement when $\Delta S > 0$; the observed positive label additionally requires no more than a five-point clean loss, which these axes cannot show. Background regions mark agreement.

The sign of ΔS agrees with the observed label on 22 of 24 pairs (0.889 balanced accuracy, 0.882 precision, 1.000 recall; balanced accuracy is 0.875 for blur and 0.900 for resize alone), and the continuous score change has Spearman's $\rho = 0.835$ with the change in success rate. The two discordant cases are both PushT pairs whose success rate improves by exactly four percentage points—one point below the fixed five-point label—while the selective score improves (Appendix E). **Takeaway:** the Gaussian-calibrated score preserves relative checkpoint ordering under these two shifts at the tested severities; it is not an absolute robustness classifier.

5 Discussion and limitations

What the evidence shows. The evidence supports three conclusions. First, eight-step ACPC under recorded actions predicts the error drift bounded by Proposition 1 beyond encoder shift, one-step ACPC, and the strongest same-horizon destroyed-action control (Section 4.4). Because every control also uses eight rollout steps, rollout length alone cannot explain this gain. Second, the CEM experiment verifies the deterministic cost bound and shows that candidate-specific ACPC adds cross-task information about decision regret (Section 4.5). The gain is smaller for maximum cost movement and absent for the first action. Third, the threshold-selection procedure—not a single numerical threshold—transfers across tasks, to blur and resize, and to PLDM after per-family recalibration (Sections 4.6 to 4.8).

Scope and limitations. The results have several limits. The planner claims concern model costs and candidate selection under paired evaluations, not environment success. The adaptive-CEM guarantee also requires the candidate pools to remain aligned and stops at the first failed elite certificate. SMPR tests only the declared coordinate proxies, so a model can still make stable but incorrect predictions. Three training runs per LeWM task, and one run per PLDM setting, support consistency checks but not precise variability estimates. The full-budget CEM analysis reuses 16 histories per task. We did not collect matched runs without the diagnostic computation and therefore cannot estimate its incremental wall-clock overhead. Finally, blur and resize are each tested at one severity, and the one-source results in Section 4.6 show that calibration depends on source-task composition.

Future work. Important next steps are direct task outcomes for ACPC-informed planner interventions, multiple severities per visual shift, richer different-state proxies, and additional model families with different objectives and planner interfaces.

6 Conclusion

ACPC measures how much a same-state visual perturbation changes a world model's predicted trajectory when the actions are held fixed. Exact inequalities relate this change to prediction error and squared latent goal cost. ATR summarizes the perturbation sensitivity of a checkpoint, while SMPR rejects constant-representation collapse on the tested state-coordinate proxies. Across four tasks, ACPC under recorded actions predicts perturbation-induced error changes beyond one-step and destroyed-action controls and tracks changes in CEM costs and decisions. The calibrated ATR–SMPR thresholds transfer to tasks excluded from calibration, to blur and resize, and—after model-family recalibration—to PLDM. ACPC therefore provides a low-cost audit before deploying a frozen checkpoint behind a planner: using only the model and logged data, it detects visual sensitivity that encoder-level checks can miss.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.

[2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[3] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mido Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024.

[4] Jiaheng Chen. ATM: Action-consistency transfer matrix for diagnosing and improving latent world models, 2026.

[5] Yuxin Chen, Jianglan Wei, Chenfeng Xu, Boyi Li, Masayoshi Tomizuka, Andrea Bajcsy, and Ran Tian. Reimagination with test-time observation interventions: Distractor-robust world model predictions for visual model predictive control, 2025.

[6] Tom Dupuis, Jaonary Rabarisoa, Quoc-Cuong Pham, and David Filliat. VIBR: Learning view-invariant value functions for robust visual control. In *Proceedings of The 2nd Conference on Lifelong Learning Agents*, volume 232 of *Proceedings of Machine Learning Research*, pages 658–682. PMLR, 2023.

[7] T. W. Epps and Lawrence B. Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983.

[8] Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G. Bellemare. DeepMDP: Learning continuous latent space models for representation learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2170–2179. PMLR, 2019.

[9] Hafez Ghaemi, Eilif Benjamin Muller, and Shahab Bakhtiari. seq-JEPA: Autoregressive predictive learning of invariant-equivariant world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[10] Christopher Grimm, Andre Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

[11] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025.

[12] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[13] Nicklas Hansen and Xiaolong Wang. Generalization in reinforcement learning by soft data augmentation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 13611–13617, 2021.

[14] David Klindt, Yann LeCun, and Randall Balestriero. When does LeJEPA learn a world model?, 2026.

[15] Ilya Kostrikov, Denis Yarats, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[16] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022.

[17] Etai Littwin, Omid Saremi, Madhu Advani, Vimal Thilak, Preetum Nakkiran, Chen Huang, and Joshua Susskind. How JEPA avoids noisy features: The implicit bias of deep linear self distillation networks, 2024.

[18] Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation, 2026.

[19] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[20] Mingyu Park, Samyeul Noh, Hyun Myung, and Donghwan Lee. Zero-shot visual generalization in model-based reinforcement learning via latent consistency. OpenReview (ICLR 2026 submission), 2025.

[21] Bo-Kai Ruan, Teng-Fang Hsiao, Ling Lo, and Hong-Han Shuai. Is the future compatible? diagnosing dynamic consistency in world action models, 2026.

[22] Finn Rasmus Schäfer, Korbinian Moller, Yuan Gao, Christian Oefinger, Sebastian Schmidt, and Johannes Betz. Imagined rollouts are kinematic, not dynamic: A diagnosis of long-horizon world-model failure, 2026. RSS Robot World Model Workshop 2026.

[23] Gawon Seo, Dongwon Kim, and Suha Kwak. ACID: Action consistency via inverse dynamics for planning with world models, 2026.

[24] Yutaka Shimizu and Masayoshi Tomizuka. Bisimulation metric for model predictive control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[25] Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features, 2022.

[26] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Stress-testing offline reward-free reinforcement learning: A case for planning with latent dynamics models. In *WRL@ICLR 2025*, 2025.

[27] Leonardo F. Toso, Davit Shadunts, Yunyang Lu, Nihal Sharma, Donglin Zhan, Nam H. Nguyen, and James Anderson. Learning invariant visual representations for planning with joint-embedding predictive world models. *arXiv preprint arXiv:2602.18639*, 2026.

[28] Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint embedding vs reconstruction: Provable benefits of latent space prediction for self-supervised learning, 2025.

[29] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.

[30] Claas A. Voelcker, Anastasiia Pedan, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-Massoud Farahmand. Calibrated value-aware model learning with probabilistic environment models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 61745–61768. PMLR, 2025.

[31] Zijie Wang, Wei Zhang, Weiming Zhang, Fanqi Zhang, Xiao Tan, Yipeng Qin, and Guanbin Li. World models as group actions, 2026.

[32] Han Yan, Zishang Xiang, Zeyu Zhang, and Hao Tang. MWM: Mobile world models for action-conditioned consistent prediction, 2026.

[33] Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Mastering visual continuous control: Improved data-augmented reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[34] Amy Zhang, Rowan T. McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[35] Zhenghao Zhang, Yuanxiang Wang, Zhenyu Guan, Yujia Yang, Bingkang Shi, Tianyu Zong, Hongzhu Yi, Guoqing Chao, Xingchen Chen, Tiankun Yang, Chenxi Bao, Tao Yu, Jingjing Zhou, and Jungang Xu. Delta-JEPA: Learning action-sensitive world models via latent difference decoding, 2026.

A Candidate-Cost and Fixed-Pool Analysis

Proof of Proposition 2. For one candidate, suppress the index and factor the squared-distance difference:

$$\begin{aligned}
|\tilde{C} - C| &= \big| \|\tilde{x} - g\|_2^2 - \|x - g\|_2^2 \big| && (19)\\
&= \big| \langle \tilde{x} - x, \tilde{x} + x - 2g \rangle \big| && (20)\\
&\le \|\tilde{x} - x\|_2 \, \|\tilde{x} + x - 2g\|_2 && (21)\\
&\le r\big(\|x - g\|_2 + \|\tilde{x} - g\|_2\big) = b. && (22)
\end{aligned}$$

The first inequality is Cauchy–Schwarz and the second is the triangle inequality. Thus, for nominal winner w and competitor j,

$$\tilde{C}_j - \tilde{C}_w \ge C_j - C_w - |\tilde{C}_j - C_j| - |\tilde{C}_w - C_w| \ge C_j - C_w - b_j - b_w.$$

For the perturbed winner $\tilde{w}$,

$$C_{\tilde{w}} \le \tilde{C}_{\tilde{w}} + b_{\tilde{w}} \le \tilde{C}_w + b_{\tilde{w}} \le C_w + b_w + b_{\tilde{w}},$$

which proves Equation (8); nonnegativity follows from the optimality of w on the nominal pool. Positivity of Equation (9) makes every perturbed competitor strictly more costly than w. The same argument for every $i \in \mathcal{E}$ and $j \notin \mathcal{E}$ proves that Equation (10) preserves the exact elite set. Strict inequalities avoid relying on implementation-specific tie breaking.

Proof of Corollary 1. At iteration zero, the proposals are identical by assumption. Common random numbers therefore produce the same ordered action pool. If the elite certificate holds, both branches select the same action vectors as elites. The CEM mean/variance update is a deterministic function of those vectors, so the next proposals are identical. Repeating this argument establishes pool and proposal alignment by induction. If a certificate fails, equality of the elite sets is unknown; the induction stops and no later shared-pool statement is made.

Relation to the weighted rollout radius. The planner bound uses the displacement at the cost readout horizon. For the weighted stack in Equation (5), $\sqrt{\alpha_H}\, r_j \leq \mathrm{ACPC}_H$ whenever $\alpha_H > 0$. The planner experiments record r_j directly, avoiding the looser $1/\sqrt{\alpha_H}$ conversion. This does not require a future target, but it does require evaluating the candidate under both matched histories.

Sharpness without additional assumptions. Neither principal bound admits a uniformly smaller right-hand side from the available quantities alone. For a unit vector u, common target $Y = 0$, and two stacked predictions au and bu with $a, b \geq 0$, the reverse-triangle bound in Equation (6) holds with equality. Likewise, setting $g = 0$, $x = au$, and $\tilde{x} = bu$ makes $|\tilde{C} - C| = |b - a|(a + b) = r(\|x - g\|_2 + \|\tilde{x} - g\|_2)$.

B Evaluation and Diagnostic Protocol

This appendix specifies the fixed evaluation and diagnostic protocol used in the main tables.

Evaluation grid. LeWM is evaluated on PushT, TwoRoom, Reacher, and Cube with evaluation seeds 42/43/44 and 100 episodes per seed. The main evaluation perturbs observation pixels with Gaussian noise at standard deviation 0.08 while keeping the goal image clean. Checkpoints trained with noise use full-sequence input-side Gaussian augmentation with $\sigma_{\max} \in \{0.01, \dots, 0.08\}$.

PLDM instantiation. PLDM uses the same four tasks, nine-checkpoint Gaussian grid, evaluation seeds, trajectory count, paired-history construction, $H = 8$ rollout statistic, and SMPR definition. For cross-task analysis, we divide ATR by the value of the corresponding unaugmented PLDM checkpoint. We then select thresholds on the other three PLDM tasks. Success rates from the evaluation task never enter threshold selection. A second test applies the corresponding LeWM threshold pair after the same within-task PLDM normalization. We do not assume that model families share raw thresholds.

Success-rate criterion. Within each task and training run, let B and M be the success rates at evaluation noise $\sigma = 0.08$ for the unaugmented checkpoint and the best checkpoint in the sweep. A checkpoint meets the success-rate criterion when its noisy-observation success rate is at least $B + 0.8(M - B)$ and its clean success rate is no more than five percentage points below that of the unaugmented checkpoint. The green spans in Figure 2 require this criterion for a majority of runs.

ATR protocol. We compute ATR separately for each task, training run, and checkpoint. Each anchor contributes one weighted radius over the rollout horizon. The default uses $H = 8$ and uniform weights $\alpha_k = 1/H$. We divide each radius by the anchor's q50 clean observed-transition scale, including the transition from the final history observation to the first future observation. We then average multiple noise draws within each anchor and take q90 across anchors. This gives the raw ATR in Equation (13), which SMPR uses as its same-state reference. For sweep plots and within-family cross-task calibration, we divide raw ATR by the corresponding unaugmented value as in Equation (16) before computing thresholds or run summaries. The numerical stabilizers are $\varepsilon_{\mathrm{norm}} = 10^{-8}$ in Equation (12) and $\varepsilon_{\mathrm{score}} = 10^{-12}$ in Equation (17). Table 4 reports how the checkpoint-level ATR reduction depends on the rollout horizon and the summary quantile.

SMPR protocol. SMPR follows Equation (15). For each eligible anchor, we find the nearest history with a different proxy label and a standardized state distance no greater than the global q35 radius. We roll both clean histories forward under the anchor's recorded actions and normalize their distance by the anchor's clean transition scale. The pair passes only if this distance is strictly greater than raw q90 ATR plus 0.10.

Table 4: Horizon and quantile sensitivity of the ATR reduction at fixed evaluation noise $\sigma = 0.08$. Each entry is the median, over the three training runs, of the ratio between the ATR of the checkpoint trained at the highest augmentation level ($\sigma_{\max} = 0.08$) and that of the unaugmented checkpoint; lower means a larger reduction. These values are descriptive and do not retune any threshold.

	$q = 0.90$ by horizon				$H = 8$ by quantile		
Task	H1	H2	H4	H8	q80	q90	q95
TwoRoom	0.05	0.04	0.05	0.06	0.06	0.06	0.06
PushT	0.06	0.07	0.06	0.09	0.07	0.09	0.09
Reacher	0.02	0.03	0.03	0.02	0.02	0.02	0.02
Cube	0.04	0.03	0.04	0.04	0.03	0.04	0.04

Table 6: Summary of the Gaussian-augmentation sweep (nine checkpoints per task: no augmentation plus eight noise levels; Figure 2 shows every level). "Best" is the level with the highest mean planning success at evaluation noise $\sigma = 0.08$; arrows give the change from the unaugmented checkpoint to that level. Relative ATR is lower-is-better, SMPR higher-is-better; the last column lists levels meeting the success-rate criterion (Section 4.1).

Task	No-aug. success (%)	Best success (%)	Best $\sigma_{\max}$	Relative ATR	SMPR	Levels meeting criterion
TwoRoom	68.8	97.1	0.08	1.00→0.07	0.11→0.98	0.01–0.08
PushT	7.2	86.8	0.06	1.00→0.11	0.28→1.00	0.03–0.08
Reacher	18.2	83.3	0.07	1.00→0.03	0.37→0.99	0.03–0.08
Cube	43.1	66.0	0.03	1.00→0.26	0.07→0.98	0.03–0.07

Table 5 makes the pairing rule explicit. The logged state is taken at the end of the observed eight-step future and standardized coordinate-wise by the dataset preprocessor. We compute all off-diagonal Euclidean distances in that task's full standardized state vector and use their global q35 as the neighborhood radius. Proxy labels use median splits computed within the fixed 100-endpoint anchor batch (anchor seed 9101); each label therefore describes the endpoint reached by that trajectory's own recorded actions. After selecting a neighbor, we roll both starting histories under the anchor's actions for the latent-distance test. We skip an anchor only when no state with a different label lies inside the neighborhood. Pairing depends only on the logged states, so the eligible counts remain constant across checkpoints and training runs.

Table 5: Implemented state-coordinate proxies for SMPR. The state is taken at the eighth observed future step and standardized coordinate-wise with full-dataset statistics. Counts give eligible and skipped anchors in the fixed 100-anchor audit; these labels are not semantic or action-relevance annotations.

Task	HDF5 key (dim.)	Implemented proxy label $\ell(y)$	Eligible / skipped
TwoRoom	`pos_agent` (2)	Median split of the standardized agent x coordinate.	61 / 39
PushT	`state` (7)	Three median bits from standardized block x, block y, and block angle (slice `state[2:5]`).	98 / 2
Reacher	`observation` (6)	Three median bits from the two standardized joint-velocity coordinates and their Euclidean norm (slice `observation[4:6]`).	100 / 0
Cube	`observation` (28)	For standardized observation $\bar{o}$, let $r = \bar{o}_{0:3} - \bar{o}_{25:28}$; use median bits of r_1, r_2, and $\|r\|_2$.	100 / 0

Threshold grids. For cross-task evaluation, candidate thresholds are $t_R \in \{0.05, 0.075, 0.10, 0.15, 0.20, 0.30\}$ for task-relative ATR and $t_M \in \{0.80, 0.85, 0.90, 0.95\}$ for SMPR. Within each source-task subset, the selection criterion lexicographically minimizes mean absolute onset error, false-early and false-late counts, then maximizes balanced accuracy. The selected pair is applied without modification to every evaluation task. No evaluation-task label or blur/resize outcome enters threshold selection.

Reproducibility. The scripts documented in `paper1/scripts/README.md` rebuild all summary figures and tables deterministically. Checkpoint-level diagnostics also require the released checkpoints and datasets. Machine-readable summaries record training seeds, evaluation seeds, and protocol hashes.

C Prediction-Error Scope and Controls

The common-future inequality in Proposition 1 has zero numerical violations in any logged eight-step or candidate-specific five-step task×run×training-condition cell; this is an implementation invariant, not an independent statistical success count.

Table 7: Relative reduction in out-of-group MAE from Base+ACPC$_8$ when predicting the error drift d on the unaugmented checkpoints (protocol and model definitions in Section 4.4). Base+Control$_8$ uses the per-cell oracle control; higher is better. The last column counts task–run cells in which Base+ACPC$_8$ is best.

Training run (seed)	vs. Base	vs. Base+Control$_8$	Task–run cells improved
3072	55.5%	51.4%	4/4
3073	60.8%	54.8%	4/4
3074	51.4%	47.7%	4/4
mean ± SD	55.9 ± 4.7%	51.3 ± 3.5%	12/12

Table 8: Out-of-group MAE (16-fold leave-one-trajectory-group-out) for predicting the error drift d on the unaugmented checkpoints; lower is better, model definitions in Section 4.4. “Selected control” is the destroyed-action control chosen by the per-cell oracle; “paired wins” counts the folds (of 16) in which Base+ACPC$_8$ beats Base+Control$_8$.

Task	run (seed)	Base	Base +Control$_8$	Base +ACPC$_8$	selected control	paired wins
TwoRoom	3072	2.535	2.099	**0.755**	shuffled actions	15/16
TwoRoom	3073	2.336	2.015	**0.866**	shuffled actions	15/16
TwoRoom	3074	2.075	1.911	**1.006**	swapped actions	13/16
PushT	3072	2.068	2.068	**1.762**	shuffled actions	11/16
PushT	3073	1.969	2.008	**1.315**	zeroed actions	13/16
PushT	3074	1.909	1.833	**1.439**	zeroed actions	13/16
Reacher	3072	1.358	1.097	**0.288**	shuffled actions	16/16
Reacher	3073	1.399	0.793	**0.247**	shuffled actions	16/16
Reacher	3074	1.409	1.136	**0.263**	shuffled actions	16/16
Cube	3072	1.171	1.037	**0.489**	zeroed actions	14/16
Cube	3073	1.416	1.212	**0.500**	zeroed actions	14/16
Cube	3074	1.061	1.008	**0.552**	shuffled actions	14/16

Logged-future prediction and candidate planning are different estimands: the former predicts the error drift against an independent action-matched observed future, whereas the latter compares candidates from the actual planner pool. We do not infer one result from the other; Section 4.5 supplies the planner-facing evidence with same-pool, candidate-specific features.

The prediction-error analysis is restricted to the unaugmented checkpoints and the listed visual probes. Every training run improves on all four tasks.

D Cross-Task Threshold Partitions

The 14 rows below are exhaustive: selecting one, two, or three of four tasks as sources creates $4+6+4$ directional partitions. Each evaluation task retains every training run and all nine checkpoints. The apparent sample size is therefore not inflated by treating checkpoint rows as independent observations.

E Cross-Stressor Pairs and Discordant Cases

Both discordant rows lie near the success-rate criterion: PushT run 3072 under resize and PushT run 3074 under blur each improve success rate by four percentage points, one point below the five-point criterion, while their selective scores increase. The complete table shows these cases rather than only the aggregate classification metrics.

Table 9: Cross-task evaluation of the selective ACPC rule over all 14 source/evaluation partitions. Thresholds (t_R, t_M) are selected on the source tasks and applied unchanged to all remaining tasks; metrics first average training runs within each evaluation task and then weight tasks equally. Onset error is in Gaussian-augmentation grid steps (one step is 0.01 in $\sigma_{\max}$).

Source tasks	Evaluation tasks	t_R	t_M	BA	P / R	Onset error (steps) mean / max
TwoRoom	PushT, Reacher, Cube	0.3	0.95	0.888	0.884 / 0.981	0.3 / 1.0
PushT	TwoRoom, Reacher, Cube	0.3	0.95	0.894	0.902 / 0.956	0.6 / 1.0
Reacher	TwoRoom, PushT, Cube	0.1	0.95	0.723	0.906 / 0.540	2.9 / 5.0
Cube	TwoRoom, PushT, Reacher	0.3	0.95	0.913	0.950 / 0.938	0.6 / 1.0
TwoRoom, PushT	Reacher, Cube	0.3	0.95	0.874	0.853 / 1.000	0.3 / 1.0
TwoRoom, Reacher	PushT, Cube	0.3	0.95	0.887	0.873 / 0.972	0.3 / 1.0
TwoRoom, Cube	PushT, Reacher	0.3	0.95	0.903	0.925 / 0.972	0.3 / 1.0
PushT, Reacher	TwoRoom, Cube	0.3	0.95	0.896	0.901 / 0.935	0.7 / 1.0
PushT, Cube	TwoRoom, Reacher	0.3	0.95	0.912	0.952 / 0.935	0.7 / 1.0
Reacher, Cube	TwoRoom, PushT	0.3	0.95	0.926	0.972 / 0.907	0.7 / 1.0
TwoRoom, PushT, Reacher	Cube	0.3	0.95	0.858	0.802 / 1.000	0.3 / 1.0
TwoRoom, PushT, Cube	Reacher	0.3	0.95	0.889	0.905 / 1.000	0.3 / 1.0
TwoRoom, Reacher, Cube	PushT	0.3	0.95	0.917	0.944 / 0.944	0.3 / 1.0
PushT, Reacher, Cube	TwoRoom	0.3	0.95	0.935	1.000 / 0.869	1.0 / 1.0

Table 10: Transfer of the Gaussian-calibrated selective score to blur and resize. For each task, thresholds are selected on the other three Gaussian-noise tasks and fixed; each pair compares the $\sigma_{\max} = 0.08$ checkpoint with its unaugmented counterpart. "Discordant" counts pairs whose score-change sign disagrees with the observed success-rate label (Section 4.8).

Visual shift	Pairs	BA	Precision / recall	Spearman	Discordant
All pairs	24	0.889	0.882 / 1.000	0.835	2
Blur	12	0.875	0.889 / 1.000	0.873	1
Resize	12	0.900	0.875 / 1.000	0.746	1

Table 11: All 24 LeWM checkpoint pairs evaluated under blur and resize. $\mathrm{ATR}_{\mathrm{rel}}$ and SMPR are measured for the checkpoint trained with Gaussian augmentation ($\sigma_{\max} = 0.08$); ΔP and ΔS are its success-rate and selective-score changes relative to the unaugmented checkpoint. A pair is positive when $\Delta P \geq 5$ percentage points with at most a five-point clean loss. Daggers mark the two discordant pairs.

Task	Seed	Shift	$\mathrm{ATR}_{\mathrm{rel}}$	SMPR	ΔP	ΔS	Outcome (success / score)
TwoRoom	3072	blur	0.499	0.885	55.7	1.670	positive / positive
TwoRoom	3072	resize	0.336	0.967	52.3	2.214	positive / positive
TwoRoom	3073	blur	0.781	0.262	36.0	0.729	positive / positive
TwoRoom	3073	resize	0.691	0.361	40.0	1.030	positive / positive
TwoRoom	3074	blur	0.636	0.541	37.7	1.212	positive / positive
TwoRoom	3074	resize	0.617	0.656	34.3	1.277	positive / positive
PushT	3072	blur	0.943	0.939	10.7	0.189	positive / positive
PushT	3072	resize	0.814	0.969	4.0	0.620	nonpositive / positive†
PushT	3073	blur	0.846	0.918	7.3	0.515	positive / positive
PushT	3073	resize	0.682	0.969	24.3	1.060	positive / positive
PushT	3074	blur	0.781	0.959	4.0	0.729	nonpositive / positive†
PushT	3074	resize	1.173	0.939	-19.7	-0.577	nonpositive / nonpositive
Reacher	3072	blur	0.155	0.990	50.7	2.375	positive / positive
Reacher	3072	resize	0.210	0.990	29.7	2.375	positive / positive
Reacher	3073	blur	0.176	0.990	52.3	2.375	positive / positive
Reacher	3073	resize	0.207	0.990	40.0	2.375	positive / positive
Reacher	3074	blur	0.190	0.990	44.7	2.375	positive / positive
Reacher	3074	resize	0.195	0.990	33.3	2.375	positive / positive
Cube	3072	blur	1.447	0.330	-2.7	-1.488	nonpositive / nonpositive
Cube	3072	resize	1.166	0.530	1.0	-0.552	nonpositive / nonpositive
Cube	3073	blur	1.577	0.260	2.3	-1.923	nonpositive / nonpositive
Cube	3073	resize	1.218	0.560	-1.3	-0.728	nonpositive / nonpositive
Cube	3074	blur	1.220	0.660	-3.0	-0.735	nonpositive / nonpositive
Cube	3074	resize	1.040	0.770	-2.3	-0.134	nonpositive / nonpositive

F Local Gaussian Sensitivity as a Mechanistic Interpretation

For a small isotropic observation perturbation $\delta x \sim \mathcal{N}(0, \sigma^2 I)$, first-order linearization of encoder E followed by rollout map G gives

$$\mathbb{E}\big[\|\Delta G\|_2^2\big] \approx \sigma^2 \|J_G J_E\|_F^2. \tag{23}$$

This explains one mechanism by which Gaussian noise training can contract ACPC: it reduces the composed local sensitivity of the encoder–predictor path. It is a local approximation, not a global robustness bound.

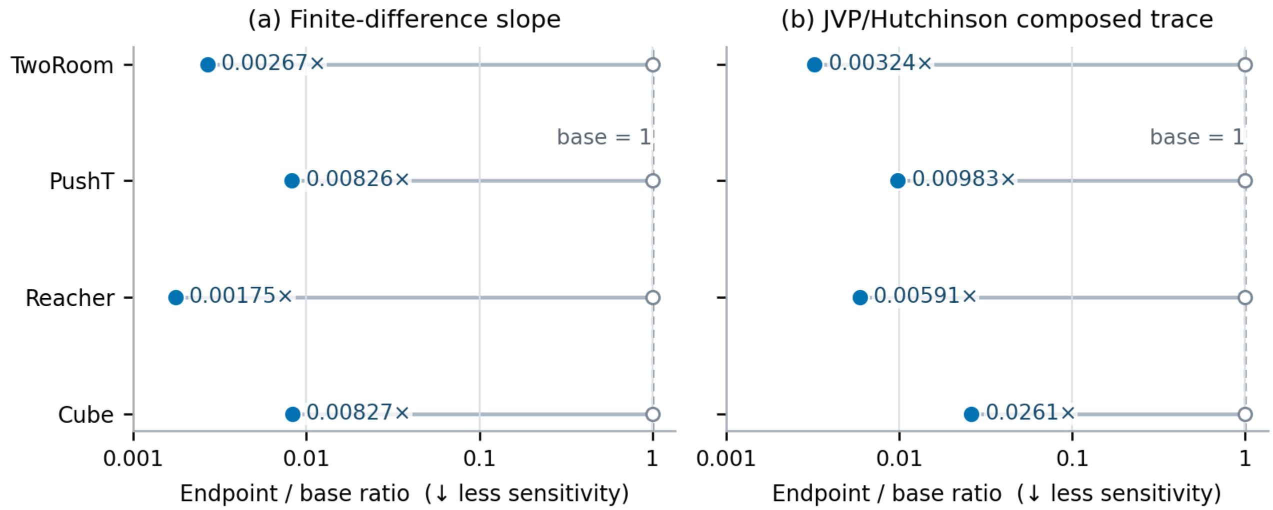

Figure 7: Local-sensitivity ratio of the checkpoint trained with Gaussian augmentation to the unaugmented checkpoint, estimated by finite differences and by a JVP/Hutchinson estimate of the composed Jacobian trace. Values below one indicate contraction after augmentation. The two estimators agree in direction for every task; these ratios do not establish task performance.

G Fixed-Pool Candidate Stability

The CEM analysis does not use task-success labels. It compares checkpoints unaugmented and $\sigma_{\max} = 0.08$ checkpoints on all four tasks. Fixed-pool shards capture the ordered initial proposal, evaluate the nominal and perturbed histories through one shared cost path, and record exact same-winner and elite-set events for 100 histories per task, run, and training condition at severities $0, 0.02, 0.05, 0.08$. Adaptive shards use a common initial proposal and common random numbers. Proposal alignment is asserted only while all preceding elite updates remain certified.

For nominal final plan $\mathbf{a}$ and perturbed plan $\tilde{\mathbf{a}}$, the reported positive clean-history regret is

$$\big[C_h(\tilde{\mathbf{a}}) - C_h(\mathbf{a})\big]_+.$$

It measures decision degradation in the model's latent goal cost, not an environment outcome. The RMS change in the first planned action is reported separately. The full-budget evaluation keeps the same definitions but uses 16 fixed histories per task shared across training conditions and runs, $K = 300$, top-30 elites, and 30 CEM steps. Whole-shard runtimes and forward-call counts are retained for provenance, but without matched timing runs that omit the diagnostic they do not estimate incremental overhead.

Across 9,600 joined reduced-budget rows there are zero squared-distance-bound violations, zero false top-1 certificates, and zero false elite certificates. Identity probes have zero five-step displacement and zero RMS change in the first planned action; all identity updates remain aligned. These checks establish implementation soundness and are not additional independent statistical samples.

At severity 0.08, the sufficient top-1 certificate covers 10–70% of histories from checkpoints trained with augmentation across task–run blocks (and none from the unaugmented checkpoints); elite-set coverage is 0–30% with augmentation. Coverage is lower than realized stability because these conditions are sufficient but not necessary.

Table 12: CEM decision statistics at Gaussian perturbation severity 0.08. Values are mean ± standard deviation across training runs after averaging histories within each run. Reduced-budget rows use 100 histories, $K = 64$, top-8 elites, and eight CEM iterations; full-budget rows use the same 16 histories per task with $K = 300$, top-30 elites, and 30 iterations. Regret uses the model's squared latent goal cost, not an environment success rate.

	Best candidate unchanged		Reduced-budget regret		Full-budget regret	
Task	No aug.	$\sigma_{\max}$=0.08	No aug.	$\sigma_{\max}$=0.08	No aug.	$\sigma_{\max}$=0.08
TwoRoom	0.13±0.03	0.93±0.02	203.81±9.97	8.47±2.75	224.97±30.56	1.75±1.44
PushT	0.13±0.09	0.92±0.01	167.50±29.53	4.30±0.28	220.14±32.97	7.91±4.23
Reacher	0.07±0.01	0.96±0.04	281.12±20.24	0.47±0.14	269.44±22.21	0.28±0.20
Cube	0.02±0.02	0.92±0.03	255.72±12.28	1.31±0.11	265.03±28.08	0.63±0.22
Equal-task mean	0.09±0.03	0.93±0.01	227.04±16.92	3.64±0.68	244.89±27.08	2.65±1.43